

Drift Reference Manual v1.0.0

README

Drift

Stack

Repo layout

Get started

License

CHANGELOG

Changelog

[Unreleased] - v0.1.0

Commit history (since repo creation)

DESIGN

Drift — DESIGN

1. Product principles
2. Repo map
3. Layered architecture
4. Discovery layers
5. Smart-reply pipeline
6. Realtime
7. Why these tech choices

DRIFT-FEATURES

Drift — FEATURE INVENTORY (v1.0)

In v1

Explicitly NOT in v1

OBSERVABILITY

Drift — OBSERVABILITY

1. Stack
2. Event taxonomy (canonical)
3. Surface
4. Dashboards
5. Sentry release naming
6. Logging hygiene

PRIVACY

Drift — PRIVACY

1. What we collect
2. What we explicitly do NOT collect
3. Verification image lifecycle
4. Telemetry opt-ins
5. User rights
6. Where it lives
7. Children
8. Legal & contact

PRODUCTION

Drift — PRODUCTION

1. Environments
2. Backend infra (Supabase)
3. Capacity & scaling
4. Incident response
5. Backups
6. Watch surfaces — minimalism by design
7. Audit log

QUICKSTART

Drift — QUICKSTART

Prereqs

1. Clone
2. Local backend
3. iOS Simulator
4. watchOS Simulator
5. Android emulator
6. Wear OS emulator
7. Run every test

Useful one-liners

RELEASE-VIDEO-SCRIPT

Release Video Script — Drift

Hero — top of marketing site | 4

Features overview | 5

Highlight reel | 5

What's next | 5

Call to action — bottom of page | 4

RELEASING

Drift — RELEASING

1. Branch + tag scheme
2. Cutting a release
3. What is uploaded where
4. Required GitHub Secrets
5. Release dry-run
6. Rolling back

SAFETY

Drift — SAFETY

1. Verification (live-selfie + reference photo)
2. Reporting + blocking
3. Anonymous display name
4. Location fuzzing math
5. Public-meetup framing
6. Screenshot detection (best-effort)
7. Bot / spam mitigation
8. Erase-all-data + delete-account
9. Operator playbook

SENTRY

Drift — SENTRY

1. Pattern: `canImport` -gated stub + transport
2. Adding the SDK in production
3. Opt-in toggle
4. PII scrubbing
5. Release tagging

SIGNING

Signing — Drift

Android

iOS

Desktop

watchOS

STORE-PACKAGING

Drift — STORE PACKAGING

1. iOS — Xcode targets vs App Store layout
2. Android — single AAB, single Wear APK
3. Bundle ID map (canonical)
4. Permissions / usage strings
5. App Store / Play Store review notes
6. Marketing site
7. What's-new copy

Desktop binaries (Electron)

TESTING

Drift — TESTING

1. Test gate matrix
2. The four pure-logic keystone tests (XCTest **and** JUnit, mirrored)
3. Manual checks
4. Coverage targets

AMERICAN GROUP LLC

Drift

Reference Manual

Version 1.0.0

Generated 2026-05-08

README

README.md

Drift

Meet people at your pace, in your orbit.

A layered dating platform where people choose how they want to be discovered: **Server, State, County, or ZIP**. Every chat opens with three reply suggestions — Casual, Context, Playful — so nobody stalls in the blank textbox. Verification before chat unlocks. Public-meetup framing instead of private location sharing.

	iPhone	Apple Watch	Android	Wear OS
Discover (4 layers)	✓	layer indicator	✓	layer indicator
Wave / Match	✓	wave-back tile	✓	wave-back tile
Smart-reply chat	✓	quick-reply	✓	quick-reply
Selfie verification	✓	—	✓	—
Voice-prompt 30s clip	✓	—	✓	—
Notifications	rich push	complication	FCM	tile

Stack

- **Apple:** Swift / SwiftUI, iOS 17, watchOS 10, XcodeGen.
- **Android:** Kotlin / Jetpack Compose, multi-module Gradle (`:core` + `:app` + `:wear`), Hilt, Room, DataStore.
- **Backend:** Supabase (Postgres + Auth + Storage + Realtime + Edge Functions). PostGIS for fuzzed-location centroid math; Row-Level Security on every table. Edge Functions in Deno/TypeScript.
- **Shared logic:** `DriftCore` Swift Package (Apple) and `:core` JVM Kotlin module (Android). Same models, mirrored case-for-case tests.

Repo layout

```
backend/           # Supabase config, migrations, Edge Functions, seed
shared/DriftCore  # Apple-side shared models + domain helpers + tests
ios/              # iPhone app + Notification Service Extension
watchos/          # Apple Watch app + WidgetKit complication
android/          # :core JVM + :app phone + :wear smartwatch
.github/workflows# 7 workflows: ci, ios, android, marketing, backend, pre-release-tests,
release
scripts/          # local dev helpers (test-all, run-*-emulator, seed-supabase)
distribution/     # release whatsnew text per locale
```

See [DESIGN.md](#) for the full architecture, [SAFETY.md](#) for verification + reporting + location-fuzzing math, and [DRIFT-FEATURES.md](#) for the feature inventory.

Get started

```
## Local backend (requires Supabase CLI)
./scripts/seed-supabase.sh

## iOS Simulator
./scripts/run-ios-sim.sh

## Android emulator
./scripts/run-android-emulator.sh

## Run every test suite
./scripts/test-all.sh
```

License

MIT — see [LICENSE](#).

CHANGELOG

CHANGELOG.md

Changelog

All notable changes to this project will be documented in this file.

This project adheres to [Semantic Versioning](#).

[Unreleased] - v0.1.0

Initial public release. See [distribution/whatsnew/v0.1.0/en-US.txt](#) for the user-facing summary.

Commit history (since repo creation)

- 3c1bf29 Initial commit (Srikanth Patchava, 2026-05-05)
- 2dde228 feat(repo): Phase 0 – repo skeleton, 12 docs, marketing site, 7 workflows, Drift palette (Srikanth Patchava, 2026-05-06)
- 6f5a0c3 feat(backend): Phase 1 – Supabase config, schema, RLS helpers, Realtime, 3 Edge Functions, Deno tests, seed (Srikanth Patchava, 2026-05-06)
- 4d3bb92 feat(shared): Phase 2 – DriftCore Swift Package: models, LayerScorer, ToneClassifier, LocationFuzzer, ReplyPromptBuilder, SupabaseClient, observability stubs, full XCTest coverage (Srikanth Patchava, 2026-05-06)
- e32f59d feat(android-core): Phase 3 – :core models, domain helpers, Ktor SupabaseClient, JUnit/Truth tests mirrored case-for-case (Srikanth Patchava, 2026-05-06)
- f976fe7 feat(ios): Phases 4-8 – DriftApp, onboarding, Discover/Wave/Matches, Chat+ReplySuggestions, Profile, Settings, Safety, services, NotificationService extension, tests (Srikanth Patchava, 2026-05-06)
- 351e5b5 feat(watchos): Phase 9 – DriftWatchApp, MatchesListView, QuickReplyView, MatchesComplication (WidgetKit) (Srikanth Patchava, 2026-05-06)
- 23b53f1 feat(android-app): Phase 10 – DriftApplication, MainActivity, RootNav, Discover/Matches/Chat/Profile/Settings Compose screens, AppModule (Hilt), FCM stub, Compose smoke test (Srikanth Patchava, 2026-05-06)
- 32cd0cf feat(wear): Phase 11 – DriftWear MainActivity, MatchTileService, QuickReplyComplicationService (Guava pinned for tile Futures) (Srikanth Patchava, 2026-05-06)
- 637e7fd fix(core): SupabaseClient compile errors – use URLBuilder.parameters.append in rawGet, expose client via @PublishedApi for inline postJson (Srikanth Patchava, 2026-05-06)
- ff573a7 fix(core-tests): align LayerScorer low-score threshold with documented weights and skip Companion field in LocationFuzzer reflection (Srikanth Patchava, 2026-05-06)
- e0c8eef fix(android-app): expose ktor-client-core via api so :app can construct SupabaseClient; mark RootNav local item() composable (Srikanth Patchava, 2026-05-06)
- a194f5a fix(wear): use Wear Compose Material (not material3) for MainActivity (Srikanth Patchava, 2026-05-06)
- 3718419 fix(wear): use ComponentActivity so setContent's @Composable lambda is in scope (Srikanth Patchava, 2026-05-06)
- 25be188 docs(distribution): add SUBMISSION-CHECKLIST.md (Phase 3 production readiness) (Srikanth Patchava, 2026-05-06)

DESIGN

DESIGN.md

Drift — DESIGN

1. Product principles

1. **Layers, not “swipes.”** The user picks the radius they want to be discovered in: Server (everyone), State, County, or ZIP-prefix. The same profile may show in multiple layers; the layer chip on the card tells you which layer surfaced it.
2. **Verification before chat unlocks.** No conversation happens before a live-selfie + reference-photo match. The reference is one of the user’s six profile photos.
3. **No blank textbox.** Every chat screen-entry calls a server-side LLM via an Edge Function and surfaces three suggested first messages: **Casual, Context, Playful.**
4. **Public-meetup framing, not location share.** When the conversation tone becomes meetup-ready, suggestions point at coffee shops / public events, never “send my pin.”
5. **Calm.** No haptic dopamine bait, no aggressive notifications, no super-like paywall.

2. Repo map

```

DriftDate/
├── README.md
├── DESIGN.md (you are here)
├── DRIFT-FEATURES.md / SAFETY.md / TESTING.md / PRIVACY.md / ...
├── codecov.yml / .gitignore / LICENSE
├── index.html / styles.css / script.js / robots.txt / sitemap.xml
├── distribution/whatsnew/ ...
├── .github/workflows/ 7 GitHub Actions workflows
│   ├── ci.yml unit tests on every push
│   ├── ios.yml iOS / watchOS sim build + tests
│   ├── android.yml Android instrumented Compose tests
│   ├── marketing.yml GitHub Pages deploy of marketing site
│   ├── backend.yml Supabase migration + Edge Function tests
│   ├── pre-release-tests.yml release gate
│   └── release.yml tag-push → all binaries + GitHub Release
├── scripts/ local dev helpers
├── backend/supabase/ Postgres schema + RLS + Edge Functions
│   ├── config.toml
│   └── migrations/0001_init.sql profiles / photos / waves / conversations /
messages / reports
│   ├── migrations/0002_realtime.sql Realtime publications
│   └── migrations/0003_rls_helpers.sql is_layer_match / can_view_profile /
can_send_message
│   ├── functions/reply-suggest/ Edge Function: 3 reply suggestions
│   ├── functions/verify-selfie/ Edge Function: AWS Rekognition CompareFaces
│   ├── functions/fuzz-location/ Edge Function: lat/lon → ZIP-prefix3
│   └── seed/seed.sql 20 demo profiles for local dev
├── shared/DriftCore/ Apple Swift Package (iOS + watchOS)
│   ├── Package.swift
│   └── Sources/DriftCore/Models/ Profile, Photo, Wave, Conversation, Message, Tone,
Intent, Layer, ReplySuggestion
│   └── Sources/DriftCore/Domain/ LayerScorer, ToneClassifier, LocationFuzzer,
ReplyPromptBuilder
│   ├── Sources/DriftCore/Networking/ thin URLSession SupabaseClient
│   ├── Sources/DriftCore/Observability/ Sentry + PostHog stubs
│   └── Tests/DriftCoreTests/ XCTest, mirrored case-for-case with Android
├── ios/ iPhone app
│   ├── project.yml XcodeGen spec
│   ├── Drift/App/ DriftApp + rootView
│   ├── Drift/Features/{Onboarding,Discover,Matches,Chat,Profile,Settings,Safety}
│   └── Drift/Services/ AuthService, ProfileService, ChatService,
DiscoverService, ReplyService, VerificationService, LocationService, PushService
│   └── Drift/Resources/ Info.plist, Drift.entitlements, Assets,
LaunchScreen
│   ├── DriftNotificationService/ rich-push extension (decrypts previews)
│   ├── DriftTests/ view-model glue tests
│   └── DriftUITests/ XCUITest smoke
├── watchos/ Apple Watch app
│   ├── project.yml
│   ├── DriftWatch/App/ DriftWatchApp
│   └── DriftWatch/Views/ MatchesListView, QuickReplyView

```

```

└─ DriftWatchComplication/      WidgetKit complication (layer + unread)
└─ android/
  └─ settings.gradle.kts        rootProject.name = Drift, includes :core, :app,
:wear
  └─ build.gradle.kts           Kotlin 2.0, Hilt, Compose plugins
  └─ core/                       pure-Kotlin/JVM, no Android deps - :core:test (NOT
testDebugUnitTest)
  └─ src/main/java/com/americangrouponllc/drift/core/{models, domain, networking, obs}/
  └─ app/                         phone app, applicationId com.americangrouponllc.drift
  └─ wear/
src/main/java/com/americangrouponllc/drift/{auth, onboarding, discover, chat, matches, profile, set
  └─ wear/                         applicationId com.americangrouponllc.driftwear
src/main/java/com/americangrouponllc/driftwear/{matches, quickreply, tile, complication}/

```

3. Layered architecture

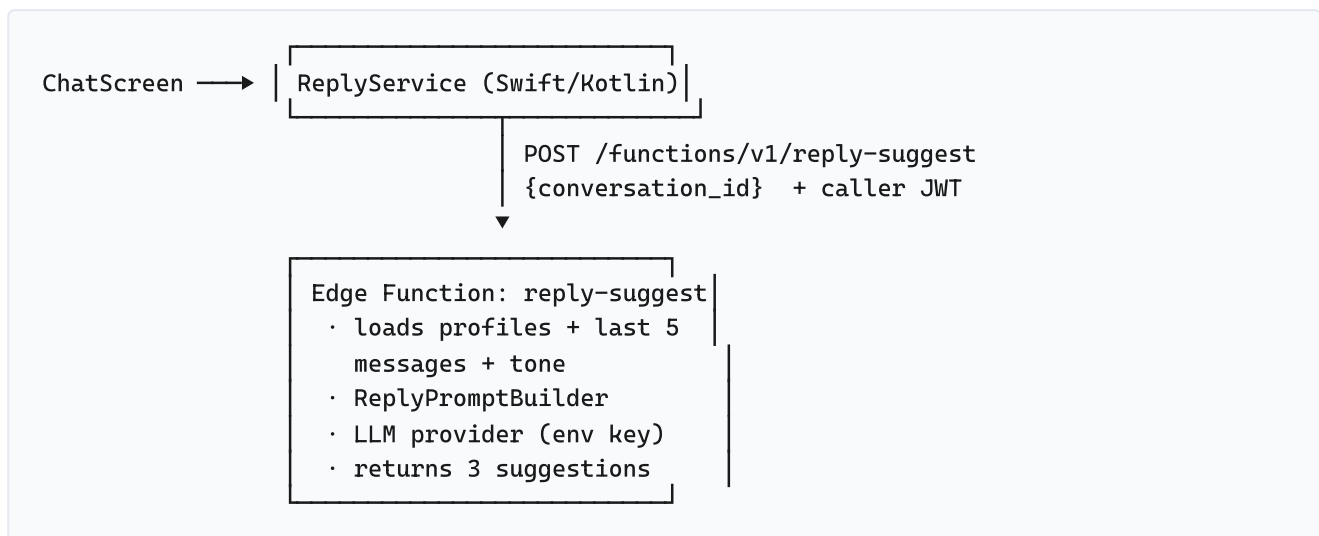
Presentation	SwiftUI (iOS + watchOS) Jetpack Compose (phone + Wear)
Services	AuthService, ProfileService, ChatService, DiscoverService, ReplyService, VerificationService, LocationService, PushService
Domain (shared)	Profile, Wave, Conversation, Message, Layer, Intent, Tone, ReplySuggestion + LayerScorer, ToneClassifier, LocationFuzzer, ReplyPromptBuilder
Networking	thin URLSession (Swift) / Ktor (Kotlin) over Supabase REST + Realtime websocket
Backend	Postgres + RLS + PostGIS centroids Realtime LISTEN/NOTIFY Edge Functions (Deno): reply-suggest (LLM provider, env key) verify-selfie (AWS Rekognition) fuzz-location (lat/lon → ZIP-3)

4. Discovery layers

Layer	Stored on profile	Resolution to client	Pool size
Server	always discoverable	full pool	unbounded
State	state_code	"in California"	thousands
County	county_fips	"in Santa Clara County"	hundreds
ZIP-3	zip_prefix3	"same ZIP-3"	tens

The **client never sees a precise lat/lon**. The on-device `LocationFuzzer` truncates to ZIP-3 before any network call. The Edge Function `fuzz-location` exists as a defence-in-depth for a buggy client; the server also strips lat/lon if it ever arrives.

5. Smart-reply pipeline



The LLM provider key is held as a Supabase secret; the client only ever sends `conversation_id`. The function fetches messages itself using the caller's JWT, so RLS still gates everything.

6. Realtime

Postgres LISTEN/NOTIFY → Supabase Realtime → websocket → client. The publication declared in `0002_realtime.sql` includes `messages`, `waves`, `conversations`. Popular profiles use a `wave_aggregates` materialized view to avoid backpressure (clients subscribe to the aggregate, not the raw 1000-wave/sec stream).

7. Why these tech choices

- **Native per platform** so we get full SwiftUI / Compose, real Sentry / PostHog SDKs, and idiomatic watch surfaces.
- **Supabase** so a tiny solo backend gets us Auth + Postgres + Storage + Realtime + Edge Functions in one bundle, with provider-portable Postgres dialect and standard JWTs.

- **Shared :core and DriftCore** so the keystone test files (**LayerScorer** , **ToneClassifier** , **LocationFuzzer** , **ReplyPromptBuilder**) live in pure-logic modules that compile on JVM and Apple — and stay green regardless of platform UI churn.

DRIFT-FEATURES

DRIFT-FEATURES.md

Drift — FEATURE INVENTORY (v1.0)

In v1

Discovery

- Layer selector: **Server · State · County · ZIP**, switchable from the home tab.
- Layer chips on every profile card: “same ZIP”, “same county”, “100 mi away”.
- Per-layer pool sizes shown above the feed: “8 nearby on ZIP, 42 in your county, 120 in California”.

Profile

- 6 photos, one **must** be tagged the verification selfie.
- Voice prompt (30s AVAudioRecorder / MediaRecorder clip).
- Intent badge — Dating / Serious / Friendship / Open.
- Vibe tags — small fixed taxonomy (~30 entries).
- Three Hinge-style structured prompts.

Matching loop

- **Wave** or **Pass** in proximity layers (tap, no swipe).
- **Like-a-prompt** in server mode.
- Mutual Wave / Like → chat unlocks (verification-gated).
- No “super likes” paywall.

Chat

- Realtime via Supabase Realtime (Postgres LISTEN/NOTIFY).
- **Three reply suggestions on every screen entry** — Casual / Context / Playful — generated by an Edge Function (LLM provider, env-keyed).
- Tone evolves: `.slow` → light prompts; `.energetic` → playful escalation; `.deep` → thoughtful follow-up; `.meetupReady` → public-meetup suggestions. (`Conversation.tone` column updated by `ToneClassifier`.)

Safety

- Selfie verification before messaging unlocks (AWS Rekognition CompareFaces).
- Report / block / mute on every profile + chat.
- Anonymous display name until mutual Wave + verification.
- Location fuzzing: ZIP-prefix-3 / county-FIPS / state-code only.
- Public-meetup framing — no private location share.
- Screenshot disclosure on iOS + Android.

Watch surfaces (Apple Watch + Wear OS)

- Match notification complication — “*Sara matched*” + quick reply.
- Glanceable layer indicator (server / state / county / zip) + unread match count.
- Wave-back tile — 1-tap accept on a pending Wave.

Settings

- Layer toggles · intent · invisibility · pause discoverability · 12/24-hour · theme (system/light/dark) · Sentry/PostHog opt-ins · erase-all-data · account deletion.

Explicitly NOT in v1

- Bluetooth-LE / WiFi-Direct discovery → v1.1.
- Venue mode → v1.1.
- IAP / RevenueCat → v1.1.
- Voice / video calling → v2.
- Stories / live presence beyond match notifications → v2.
- Cross-platform photo / message search → v2.
- AI auto-classification of profiles → v2.
- Group / friend invites → v2.

OBSERVABILITY

OBSERVABILITY.md

Drift — OBSERVABILITY

1. Stack

Concern	Tool	Drift event surface
Crash reporting	Sentry (<code>canImport</code> -gated)	<code>CrashReportingService.shared.capture(...)</code>
Product analytics	PostHog (opt-in)	<code>AnalyticsService.shared.track(...)</code>
Backend logging	Supabase Logflare	<code>select * from logs.events where service='reply-suggest'</code>
Backend metrics	Supabase + Grafana Cloud	<code>pg_stat_*</code> , Edge Function p50/p95 latency

All client SDKs are **opt-in** by default. The user toggles them in **Settings** → **Telemetry**. The first build of a new install ships with both toggles **off** so we never collect anything from a user who hasn't seen the toggle.

2. Event taxonomy (canonical)

The same names fire from Apple and Android via the shared `AnalyticsEvent` sealed type / enum.

Event	Properties	Fired from
onboarding_completed	{phone_otp: bool, photos_count, selfie_verified}	end of onboarding
wave_sent	{layer, surface}	DiscoverScreen
wave_matched	{layer, time_to_match_seconds}	server-pushed
chat_screen_open	{conversation_id, tone}	ChatScreen on appear
reply_suggestion_used	{tone, kind: casual context playful}	ReplySuggestionsBar tap
reply_suggestion_dismissed	{tone}	user typed instead
verification_started	{}	VerificationCamera open
verification_succeeded	{similarity_pct}	Edge Function 200
verification_failed	{reason}	Edge Function 4xx/5xx
report_filed	{reason}	ReportSheet submit
block_user	{}	profile / chat menu
settings_toggled	{name, enabled}	SettingsScreen
layer_switched	{from_layer, to_layer}	DiscoverScreen tabs
app_opened_from_push	{push_type: match message}	PushService didReceive

3. Surface

The `Surface` enum (Swift) / `Surface` (Kotlin) labels which capture surface produced an event. For Drift v1:

```
.app           // main iPhone / Android app
.notificationExtension // iOS Notification Service Extension
.watch        // Apple Watch
.complication  // WidgetKit complication
.tile         // Wear OS tile
```

4. Dashboards

- **Funnel — onboarding:** `phone_otp_sent` → `phone_otp_verified` → `photos_uploaded` → `verification_succeeded` → `onboarding_completed`.
- **Funnel — first match:** `wave_sent` → `wave_matched` → `chat_screen_open` → first message sent.

- **Reply suggestions:** `reply_suggestion_used / total_reply_suggestions`.
- **Verification fail rate** by reason.
- **Edge Function p50/p95 latency** (Supabase dashboard → Functions).

5. Sentry release naming

```
drift@<MARKETING_VERSION>+<CURRENT_PROJECT_VERSION>  
e.g. drift@1.1.0+42
```

Same string on Apple and Android, so a single release marker covers both.

6. Logging hygiene

- **Never log message text.** `ChatService.send` logs only the hash and length.
- **Never log lat/lon.** `LocationService` logs only the resolved `zipPrefix3` etc.
- **Never log selfie image bytes.** Only the comparison-similarity score.
- Crash reports scrub usernames before upload via Sentry's `before-send` hook.

PRIVACY

PRIVACY.md

Drift — PRIVACY

Drift is a dating app. The privacy stakes are high. This doc is the single source of truth for what we collect, why, and how to delete it.

1. What we collect

Data	Purpose	Storage	Retention
Phone number	Auth (OTP)	Supabase Auth (encrypted at rest)	until account deletion
Email (optional)	Auth fallback	Supabase Auth	until account deletion
Display name	Shown on profile cards	Postgres <code>profiles.display_name</code>	until account deletion
Date of birth	Age range computation	Postgres <code>profiles.dob</code>	until account deletion
Profile photos (≤ 6)	Profile UI	Supabase Storage <code>photos/</code>	until account deletion
Reference selfie	Verification matching	Supabase Storage <code>selfies/</code>	deleted within 60s of Edge Function CompareFaces returning
Voice prompt ($\leq 30s$)	Profile UI	Supabase Storage <code>voice/</code>	until account deletion
Intent + vibe tags + prompts	Discovery + matching	Postgres	until account deletion
ZIP-prefix-3 / countyFips / stateCode	Discovery layers	Postgres <code>profiles.*</code>	until account deletion
Wave events	Match graph	Postgres <code>waves</code>	until account deletion
Messages	Realtime chat	Postgres <code>messages</code>	until account deletion or per-conversation deletion
Reports	Trust & Safety	Postgres <code>reports</code>	indefinite (legal hold)

2. What we explicitly do NOT collect

- **No precise lat / lon.** The on-device `LocationFuzzer` truncates before any HTTP call. The schema has no `lat` / `lon` columns.
- **No facial-feature embeddings.** AWS Rekognition CompareFaces returns a similarity float; we persist only the boolean threshold result.
- **No browsing history or contact list.**

- **No advertising IDs.** No ATT prompt on iOS, no `AdvertisingIdClient` on Android.
- **No third-party trackers in the marketing site.** Static HTML, no analytics.

3. Verification image lifecycle

1. User taps Capture on `VerificationCamera`.
2. Image is uploaded to `selfies/<userId>/<uuid>.jpg`.
3. Edge Function `verify-selfie/index.ts` is invoked with `{selfie_image_id, comparison_photo_id}` (the comparison photo is one of the user's six tagged photos).
4. Edge Function calls AWS Rekognition `CompareFaces` with similarity threshold ≥ 90 .
5. On success, `profiles.verified_at = now()` is set.
6. **The selfie image is deleted from `selfies/` immediately** (TTL = 60s).
7. **No facial-features metadata is persisted.**

This flow is documented for App Store / Play Store reviewers in `STORE-PACKAGING.md` §5.

4. Telemetry opt-ins

- Sentry crash reports — **off by default**. Toggleable in Settings → Telemetry.
- PostHog product analytics — **off by default**. Same toggle.
- Both wrappers PII-scrub before upload (`display_name`, phone, email, any lat/lon-shaped strings) — see `SENTRY.md` §4.

5. User rights

Right	How
Access (download my data)	Settings → Account → "Email me my data". A signed link to a JSON export is emailed within 24h.
Correction	Edit Profile screen.
Erasure	Settings → Account → "Erase all data". Deletes profile + photos + voice + messages immediately. Reports cannot be erased (legal hold).
Account deletion	Settings → Account → "Delete account". Closes the <code>auth.user</code> row in addition to the data wipe above.
Portability	The "Email me my data" export is JSON, machine-readable.

6. Where it lives

- **All persistent data** lives in the Drift Supabase project (US-East by default; EU project for users in EU when launched).

- **Verification selfies** transit AWS Rekognition `us-east-1` in-flight. No image bytes are persisted in AWS — only the API call result.
- **No CDN cache** of the photos URLs (signed URLs, 5-minute TTL).
- **No raw SQL access from clients** — every query goes through PostgREST with RLS enforced.

7. Children

Drift is **18+** (matches Tinder / Hinge). Onboarding asks for date of birth, refuses signups with computed age < 18, and the verification step serves as an additional liveness check.

8. Legal & contact

- Operator: American Group LLC
- Contact for privacy: privacy@americangroupllc.com
- DPO (when in EU jurisdictions): TBD before EU launch.

This document is updated atomically with the schema. Diff `PRIVACY.md` against `0001_init.sql` to see if changes are out of sync.

PRODUCTION

PRODUCTION.md

Drift — PRODUCTION

Operational notes for running Drift in production.

1. Environments

Env	Supabase project	Bundle suffix	Notes
local	<code>supabase start</code> (Docker)	<code>com.americangroupllc.drift.dev</code>	seeded with 20 demo profiles
staging	<code>drift-staging.supabase.co</code>	<code>com.americangroupllc.drift.staging</code>	TestFlight + Play Internal
production	<code>drift.supabase.co</code>	<code>com.americangroupllc.drift</code>	App Store + Play Production

Each env has its own Supabase secret bundle (`LLM_API_KEY` , AWS keys, Sentry DSNs). The clients pick the env via build-time `SUPABASE_URL` + `SUPABASE_ANON_KEY` — read from `Info.plist` on Apple, from `BuildConfig` on Android.

2. Backend infra (Supabase)

- **Postgres:** 15.x, single primary, point-in-time recovery enabled.
- **PostGIS:** enabled in `0001_init.sql` , used only for the county centroid table; no lat/lon at row level.
- **Auth:** phone OTP provider (Twilio integration in Supabase). Email OTP also enabled for accounts that signed up before phone was configured.
- **Storage:** two buckets — `photos` (public-read after RLS) and `selfies` (private, deleted after Edge Function `CompareFaces` returns).
- **Realtime:** publication declared in `0002_realtime.sql` includes `messages` , `waves` , `conversations` , `wave_aggregates` .
- **Edge Functions:** `reply-suggest` , `verify-selfie` , `fuzz-location` , `wipe-me` . Each one is deployed via `supabase functions deploy <name>` from CI.

One-time deploy required for `wipe-me` against production:

```
supabase functions deploy wipe-me --project-ref <ref>
```

Both the iOS and Android Settings screens call `POST /functions/v1/wipe-me` to satisfy the GDPR “right to erasure”. Until this function exists in the live project the clients receive a

404. The user-owner must run the deploy command once against the production project ref. See `backend/supabase/functions/wipe-me/README.md` for the function spec.

3. Capacity & scaling

- Initial sizing: Supabase Pro tier is plenty for the alpha ($\leq 10k$ MAU).
- The wave fan-out for popular profiles uses the `wave_aggregates` materialised view — clients subscribe to the aggregate channel, not the raw insert stream. Refresh interval: every 5 s, set in `0001_init.sql` via `pg_cron`.
- Edge Function cold-start budget: 800 ms p95 for `reply-suggest`. Beyond that the client falls back to canned suggestions (`ReplySuggestion.fallback`).

4. Incident response

Symptom	Triage	Mitigation
Edge Function 5xx > 1% in 5 min	Sentry release alert	Roll forward with cached canned suggestions.
Realtime websocket drops > 10%	PostHog event <code>realtime_disconnected</code>	Increase ping interval; surface "reconnecting..." in UI.
LLM provider outage	Edge Function logs	Switch <code>LLM_API_KEY</code> to fallback provider; redeploy <code>reply-suggest</code> .
Selfie-verify CompareFaces 5xx	AWS Rekognition status	Lower the similarity threshold by 1 point and queue retry.
Phone OTP cost spike	Twilio dashboard	Tighten reCAPTCHA threshold.
Sudden spike in <code>reports</code>	Supabase SQL view	Manual triage. Consider IP-based suspension.

5. Backups

- Postgres PITR is on by default at the Supabase Pro tier (7-day window).
- Storage objects are versioned for 30 days.
- The schema migrations are the source of truth — a fresh restore with `supabase db reset` always reproduces the live schema.

6. Watch surfaces — minimalism by design

The watch app is **deliberately minimal** in v1: notification, layer indicator, quick-reply tile/complication. No discover, no profile-edit, no deep chat composition.

This keeps watch test surface tiny and avoids the 30%-file-count tax for marginal user value. The plan for watch v2 (full discover) is gated on real adoption telemetry first.

7. Audit log

A read-only Postgres view `audit.user_actions` aggregates every report, block, profile edit, and verification result for ops review. Not exposed via the client REST API.

QUICKSTART

QUICKSTART.md

Drift — QUICKSTART

Local dev in one tab.

Prereqs

- macOS 14+ for iOS / watchOS work; Linux/Windows works for Android + backend.
- Xcode 16 + the iOS 17 Simulator runtime.
- Android Studio Hedgehog or newer; Android SDK 34; Wear OS API 33 system image.
- JDK 17.
- [xcodegen](#) — `brew install xcodegen`.
- [supabase CLI](#) — `brew install supabase/tap/supabase`.
- [Deno](#) ≥ 1.45 for Edge Function tests.
- (Optional) [watchman](#) for fast script reruns.

1. Clone

```
git clone git@github.com:AmericanGroupLLC/DriftDate.git
cd DriftDate
```

2. Local backend

```
./scripts/seed-supabase.sh # supabase db reset + seed 20 demo profiles
```

This boots a local Postgres + GoTrue + Realtime + Storage in Docker, runs all three migrations, and inserts the demo profiles.

3. iOS Simulator

```
./scripts/run-ios-sim.sh # generates Drift.xcodeproj via xcodegen, opens, runs
```

The first run prompts for Apple ID signing — use any free dev account. The `SUPABASE_URL` and `SUPABASE_ANON_KEY` are read from `Bundle.main.infoDictionary` and default to the local Supabase instance.

4. watchOS Simulator

```
cd watchos && xcodegen generate && open DriftWatch.xcodeproj
```


Pick the **DriftWatch** scheme and run on an Apple Watch Series 10 sim.

5. Android emulator

```
./scripts/run-android-emulator.sh # boots pixel_6 API 33 emulator  
cd android && ./gradlew :app:installDebug
```

6. Wear OS emulator

```
./scripts/run-wear-emulator.sh  
cd android && ./gradlew :wear:installDebug
```

7. Run every test

```
./scripts/test-all.sh
```

Runs DriftCore Swift Package tests, `:core:test`, `:app:testDebugUnitTest`, `:wear:testDebugUnitTest`, and Edge Function `deno test`.

Useful one-liners

```
## Bump the marketing version in every project file  
./scripts/bump-version.sh 1.0.1  
  
## Dry-run a release without pushing a tag  
./scripts/release-dry-run.sh  
  
## Link the local CLI to a remote Supabase project  
./scripts/link-supabase.sh <project-ref>
```

RELEASE-VIDEO-SCRIPT

RELEASE-VIDEO-SCRIPT.md

Release Video Script — Drift

This file drives the silent MP4 release video built by the umbrella's `release-video.yml` reusable workflow at tag time.

Each `## Heading | <duration_seconds>` defines one scene. Within the scene, list keys as bullets.

Recognised keys:

- `viewport: WIDTHxHEIGHT` (default 1280x720)
- `scroll: <px|%|top|bottom>` (default 0)
- `wait: <seconds|ms>` (dwell time after navigation)
- `caption: <text>` (placeholders `{APP_NAME}` and `{VERSION}`)
- `duration: <seconds>` (override the heading-level duration)

The intro and outro cards (3 s each) are auto-generated from the brand colour in `release.config.json`.

Tagline: Where date night meets focus.

Hero — top of marketing site | 4

- `viewport: 1280x720`
- `scroll: top`
- `wait: 600ms`
- `caption: {APP_NAME} {VERSION}`

Features overview | 5

- `viewport: 1280x720`
- `scroll: 25%`
- `wait: 600ms`
- `caption: New in {VERSION}`

Highlight reel | 5

- `viewport: 1280x720`
- `scroll: 50%`
- `wait: 600ms`
- `caption: Built for everyone`

What's next | 5

- `viewport: 1280x720`
- `scroll: 75%`

- wait: 600ms
- caption: Available everywhere

Call to action — bottom of page | 4

- viewport: 1280x720
- scroll: bottom
- wait: 800ms
- caption: Try {APP_NAME} today

RELEASING

RELEASING.md

Drift — RELEASING

1. Branch + tag scheme

- `main` is always shippable.
- Every release lives behind a tag: `vMAJOR.MINOR.PATCH` (e.g. `v1.0.0`).
- Pre-release tags use suffixes: `v1.1.0-rc1`, `v1.1.0-beta3`, `v1.1.0-alpha1`, or `v1.1.0-internal4`.
The release workflow auto-flags these as pre-releases on GitHub.

2. Cutting a release

1. **Bump version** on every platform in one commit:

```
./scripts/bump-version.sh 1.1.0
```

This updates:

- `ios/project.yml` `MARKETING_VERSION`
- `watchos/project.yml` `MARKETING_VERSION`
- `android/app/build.gradle.kts` `versionName` + `versionCode`
- `android/wear/build.gradle.kts` `versionName` + `versionCode`

2. **Update what's-new copy** in `distribution/whatsnew/whatsnew-en-US/whatsnew.txt`.

3. **Run pre-release tests locally** (or trigger the workflow):

```
gh workflow run pre-release-tests.yml --ref main
```

4. **Tag + push:**

```
git tag v1.1.0  
git push origin v1.1.0
```

5. The `release.yml` workflow runs the pre-release-tests gate, builds the four binaries (iOS .app, watchOS .app, Android phone APK + AAB, Wear APK), creates a GitHub Release with auto-generated notes, and *if the right secrets are set*, uploads to Google Play and TestFlight.

3. What is uploaded where

Artefact	Destination	Trigger
Drift-iOS-iPhone-vX.Y.Z-Simulator.app.zip	GitHub Release	always
Drift-Apple-Watch-vX.Y.Z-Simulator.app.zip	GitHub Release	always
Drift-Android-Phone-vX.Y.Z.apk / .aab	GitHub Release	always
Drift-Android-Watch-vX.Y.Z.apk	GitHub Release	always
Drift-Web-vX.Y.Z.zip (marketing site bundle)	GitHub Release	always
Signed .ipa → TestFlight	Apple App Store Connect	requires APP_STORE_CONNECT_API_KEY_P8_BASE64
.aab → Google Play	Play Console	requires PLAY_STORE_SERVICE_ACCOUNT_JSON

4. Required GitHub Secrets

The workflow degrades gracefully when these are absent (the optional upload steps are gated by `if [ -z "$X" ]`).

iOS / watchOS

- APP_STORE_CONNECT_API_KEY_ID
- APP_STORE_CONNECT_API_ISSUER_ID
- APP_STORE_CONNECT_API_KEY_P8_BASE64 — base64 of the .p8 file
- APPLE_TEAM_ID
- MATCH_PASSWORD, MATCH_GIT_URL — Fastlane Match storage
- SENTRY_DSN_IOS, POSTHOG_API_KEY_IOS, POSTHOG_HOST

Android / Wear

- ANDROID_KEYSTORE_BASE64 — base64 of upload.jks
- ANDROID_KEYSTORE_PASSWORD
- ANDROID_KEY_ALIAS
- ANDROID_KEY_PASSWORD
- PLAY_STORE_SERVICE_ACCOUNT_JSON
- PLAY_STORE_PACKAGE_NAME — usually `com.americangroupllc.drift`
- SENTRY_DSN_ANDROID, SENTRY_DSN_WEAR, POSTHOG_API_KEY_ANDROID

Backend

- `SUPABASE_PROJECT_REF` — for `backend.yml` deploy steps.
- `SUPABASE_DB_PASSWORD` — for `supabase db push`.
- `LLM_API_KEY` — set as a Supabase **secret** via `supabase secrets set`, not as a GitHub secret.
- `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_REGION` — Supabase secrets used by `verify-selfie/index.ts`.

5. Release dry-run

```
./scripts/release-dry-run.sh v1.1.0
```

Mirrors the CI build steps locally. Useful to catch packaging issues before tagging.

6. Rolling back

If a release is critically broken:

```
gh release edit vX.Y.Z --draft # un-publish from GitHub Release
## Play Store + App Store rollbacks are manual via their consoles.
```

Tag a follow-up patch release immediately rather than retracting (consumers who already pulled the .apk should be migrated forward).

SAFETY

SAFETY.md

Drift — SAFETY

Verification, reporting, abuse, location fuzzing.

1. Verification (live-selfie + reference photo)

1. During onboarding the user uploads six profile photos. They tag exactly one as the **reference selfie** (auto-suggested: most recent face-on photo per `Vision / MLKit` face detection).
2. The user opens `VerificationCamera` (live preview, head must occupy $\geq 30\%$ of frame, eyes detected). One frame is captured and uploaded as the **comparison selfie**.
3. The Edge Function `verify-selfie/index.ts` calls AWS Rekognition `CompareFaces` with similarity threshold ≥ 90 .
4. On match, `profiles.verified_at = now()`. The comparison selfie is deleted from Storage immediately; only the boolean result is persisted. No facial-feature embeddings are stored.
5. Until verification passes, **chat is locked**. The match exists, but the reply suggestions UI shows a “Verify to chat” call-to-action instead of the keyboard.

2. Reporting + blocking

Every profile and every chat surface has a **Report** and a **Block** affordance.

- `Report` writes a row in `reports (reporter_id, target_id, reason, note, created_at)`. Operators see this in a private Supabase view. Reporter sees a confirmation: *“Thanks — we’ll review within 24 hours.”*
- `Block` writes a row in `blocked_users (blocker_id, blocked_id, created_at)`. RLS prevents either party from `SELECT` -ing the other’s profile or conversations after that point.
- Mute is a per-conversation setting (`conversations.muted_by_a / muted_by_b`) that suppresses push notifications.

3. Anonymous display name

Until two users have a mutual Wave **and** verification on both sides, the name shown on the card is the chosen `display_name` (not real name). The `profiles.legal_name` column is read-only to ops, never to clients.

4. Location fuzzing math

`shared/DriftCore/Sources/DriftCore/Domain/LocationFuzzer.swift` (and the mirrored Kotlin file in `:core`) implements:

lat, lon	→	zipPrefix3	(first 3 digits of the ZIP code at that point)
	→	countyFips	(5-digit FIPS code from a static state→county polygon table)
	→	stateCode	(USPS 2-letter)

The function is **pure**: same input, same output, no I/O. The lat/lon never leaves the device:

- The client truncates **before** any HTTP call.
- The server's `fuzz-location` Edge Function exists only as a defence in depth — if a buggy client somehow sends `{lat, lon}`, the function truncates and discards.
- The Postgres schema has **no lat or lon column**. There is only a PostGIS centroid for the **county**, used by `is_layer_match()` and never exposed via REST.

5. Public-meetup framing

When `ToneClassifier` returns `.meetupReady`, `ReplyPromptBuilder` adds a system-prompt clause that biases the LLM toward suggestions like *"Coffee at Equator on University Ave Saturday?"* and away from *"send me your address."*

There is no in-app private location share. No "Tonight Mode." No "Drop a Pin." If users want to share a place they're meeting, they do it through their existing maps/messaging apps — Drift never sees those coordinates.

6. Screenshot detection (best-effort)

- iOS uses `UIScreen.userDidTakeScreenshotNotification` to surface a one-time toast: *"Screenshots may be reported. Be respectful."* This does not prevent the screenshot.
- Android has no screenshot-detection API. A one-time disclosure on first chat screen says: *"Photos and chats may be screenshot. Be careful what you share."*
- Neither platform attempts `FLAG_SECURE` — false sense of security and breaks legitimate accessibility tools.

7. Bot / spam mitigation

- Phone OTP via Supabase Auth, **per-IP and per-phone rate limit** (default Supabase config; raise limits only after measuring real volume).
- `reCAPTCHA v3` wrapper on the OTP request endpoint (gated by env `RECAPTCHA_SITE_KEY`).
- Verification gate (no chat without selfie match) raises the bar from "make 1000 fake accounts" to "make 1000 fake accounts that pass live-selfie matching against a reference photo." Not impossible, but considerably harder than the alternative.

8. Erase-all-data + delete-account

- Settings → Erase all data → confirms and calls `DELETE /functions/v1/wipe-me` which removes profile, photos, waves, messages, and conversations.
- Settings → Delete account → also closes the Supabase auth.user row.

Both are irreversible. The client also flushes its local Room/JSON cache.

9. Operator playbook

See `OBSERVABILITY.md` for dashboards.

Signal	Where	Action
Spike in <code>reports</code> per hour	Supabase SQL view	Triage by reporter trust score; suspend repeat offenders.
Failed selfie-verification rate > 30%	PostHog event	Check Rekognition latency; lower threshold by 1 point if false-negative spike.
Edge Function <code>reply-suggest</code> 5xx > 1%	Sentry	Provider outage; serve canned suggestions.
Phone OTP cost > budget	Twilio dashboard	Tighten reCAPTCHA threshold; add <code>accept-language</code> heuristics.

SENTRY

SENTRY.md

Drift — SENTRY

How Drift wires Sentry without a hard SDK dependency.

1. Pattern: `canImport` -gated stub + transport

Both Apple and Android keep crash reporting behind a thin interface (`CrashReportingService`) and a swappable `CrashTransport` . The Sentry SDKs are loaded conditionally:

- **Swift:** `#if canImport(Sentry) ... #else ... #endif` in `shared/DriftCore/Sources/DriftCore/Observability/CrashReportingService.swift` .
- **Kotlin:** `if (Class.forName("io.sentry.Sentry") != null)` reflective load in `android/core/src/main/java/com/americanagroupllc/drift/core/obs/CrashReportingService.kt` .

This keeps:

- Local builds green even if a contributor hasn't pulled the SDK.
- The release `xcodebuild` / Gradle pipelines free to add the SDK in CI without changing source.
- Open-source forks completely free of Sentry artefacts.

2. Adding the SDK in production

Apple

`shared/DriftCore/Package.swift` adds the dependency conditionally only in CI builds via an environment-driven `package.appendIfRelease` . To add it locally:

```
.package(url: "https://github.com/getsentry/sentry-cocoa", from: "8.30.0"),
```

...and `.target(name: "DriftCore", dependencies: ["Sentry"])` .

Android

`android/core/build.gradle.kts` :

```
implementation("io.sentry:sentry-android:7.13.0")
```

The release `assembleRelease` step in `release.yml` reads `SENTRY_DSN` from env, sets it as a `BuildConfig` field, and the `CrashReportingService.attach( ... )` is called from the `DriftApplication` `onCreate` *only when DSN is non-empty*.

3. Opt-in toggle

`SettingsScreen` exposes:

```
[ ] Send crash reports (Sentry)
[ ] Send anonymous product analytics (PostHog)
```

The toggles set `CrashReportingService.shared.optedIn` and `AnalyticsService.shared.optedIn`. When false, the transport is held but no events are forwarded.

4. PII scrubbing

- Sentry `beforeSend` hook strips `display_name`, `phone_number`, `email`, `lat`, `lon` from event tags / extras / breadcrumbs.
- We never put a user's `display_name` in an exception message — use `profileId` instead.
- Stack traces are otherwise verbatim.

5. Release tagging

Release names follow the convention from `OBSERVABILITY.md` § 5: `drift@<MARKETING_VERSION>+<CURRENT_PROJECT_VERSION>`. The CI uploads debug symbols (dSYMs / mapping files) to Sentry per release.

SIGNING

SIGNING.md

Signing — Drift

This document lists every secret used by the release pipeline and the exact code-signing wiring for each platform. Workflows degrade gracefully when secrets are absent (Android → unsigned APK; iOS → unsigned Simulator `.app.zip`; Desktop → unsigned binaries).

See the canonical secret reference in the umbrella repo:

<https://github.com/AmericanGroupLLC/AmericanGroupLLC/blob/main/SECRETS.md>

Android

Secret	Required	Purpose
ANDROID_KEYSTORE_BASE64	optional	Base64-encoded <code>upload.jks</code>
ANDROID_KEYSTORE_PASSWORD	optional	Keystore password
ANDROID_KEY_ALIAS	optional	Key alias
ANDROID_KEY_PASSWORD	optional	Key password
PLAY_STORE_SERVICE_ACCOUNT_JSON	optional	Google Play upload
PLAY_STORE_PACKAGE_NAME	optional	Play Console package

A template is at `keystore.properties.example`.

iOS

The build-ios job takes one of two paths based on whether `APPLE_CERTIFICATE_BASE64` is set:

- **Set** → import `.p12` to a temp keychain, install the provisioning profile, archive `-sdk iphoneos`, generate `exportOptions.plist` from `release.config.json.bundleId` (overridable via `APPLE_BUNDLE_ID`), `xcodebuild -exportArchive`, attach `<AppSlug>-iOS-<version>.ipa` to the GitHub Release.
- **Unset** → unsigned `-sdk iphonesimulator` build attached as `<AppSlug>-iOS-<version>-Simulator.app.zip` (Simulator-only, cannot run on real devices).

Secret	Required	Purpose
APPLE_CERTIFICATE_BASE64	optional	.p12 (Distribution)
APPLE_CERTIFICATE_PASSWORD	optional	.p12 password
APPLE_KEYCHAIN_PASSWORD	optional	Temp keychain pw
APPLE_PROVISIONING_PROFILE_BASE64	optional	Wildcard mobileprovision (recommended for DriftNotificationService)
APPLE_TEAM_ID	optional	10-char team ID
APPLE_BUNDLE_ID	optional	Override the default bundle id (<code>com.americangroupllc.drift</code>)
APP_STORE_CONNECT_API_KEY_ID	optional	TestFlight upload
APP_STORE_CONNECT_API_ISSUER_ID	optional	TestFlight upload
APP_STORE_CONNECT_API_KEY_P8_BASE64	optional	TestFlight upload

Provisioning profile shape

The default bundle ID is `com.americangroupllc.drift`. Because this app ships with extensions (`DriftNotificationService`), use a **wildcard** profile such as `com.americangroupllc.drift.*` so the profile covers the host app and every extension target. App Store Connect requires explicit IDs for each target, but the *upload step* is gated by a separate secret and only runs after the .ipa is built.

Desktop

The desktop build job runs on Ubuntu and produces:

- Windows NSIS installer `*.exe` (signed only if you add Windows Authenticode in a follow-up — out of scope here).
- Linux AppImage (unsigned, portable, no install required).
- macOS .dmg (**unsigned**; Gatekeeper will block on first launch. See `STORE-PACKAGING.md` for the `xattr -cr` bypass and the notarization roadmap).

watchOS

The release job builds watchOS as a **standalone Simulator .app.zip** for QA. Submitting a watch app to the App Store requires the watch target to be embedded inside the iOS `.ipa` (watch2-app pattern), which kicks in automatically once `APPLE_CERTIFICATE_BASE64` is set and the Xcode project's iOS target depends on the watch target.

STORE-PACKAGING

STORE-PACKAGING.md

Drift — STORE PACKAGING

Notes on getting Drift into the App Store, Play Store, and TestFlight / Play Internal track.

1. iOS — Xcode targets vs App Store layout

XcodeGen generates a project with these targets:

- `Drift` (the iPhone app)
- `DriftNotificationService` (Notification Service Extension, rich-push)
- `DriftTests`
- `DriftUITests`

The Apple Watch app is in **a separate project** (`watchos/DriftWatch.xcodeproj`). For development this is fine — both run side by side in the simulator.

For App Store submission, the watchOS app must be embedded inside the iOS app's `.ipa` as a watch2-app extension target. This requires either: 1. Combining `watchos/project.yml` into `ios/project.yml` as a nested target with the `WKApplication` flag, **or** 2. Using `xcodebuild -workspace` with the watchOS project added as a sub-project.

Until Drift submits to App Store, we keep them split for faster incremental development. The migration is a single `ios/project.yml` change planned for v1.1.

2. Android — single AAB, single Wear APK

Two `applicationIds` — `com.americangroupllc.drift` and `com.americangroupllc.driftwear` — meaning **two separate Play Console listings** linked together (Play recognises a “wearable companion” via the `<uses-feature android:name="android.hardware.type.watch" />` flag in the wear manifest).

3. Bundle ID map (canonical)

Surface	Bundle ID
iOS app	<code>com.americangroupllc.drift</code>
iOS Notification Extension	<code>com.americangroupllc.drift.notify</code>
Apple Watch app	<code>com.americangroupllc.drift.watchkitapp</code>
Apple Watch complication	<code>com.americangroupllc.drift.complication</code>
Android phone	<code>com.americangroupllc.drift</code>
Wear OS app	<code>com.americangroupllc.driftwear</code>
iOS App Group	<code>group.com.americangroupllc.drift</code>

4. Permissions / usage strings

iOS — `Info.plist` keys

- `NSCameraUsageDescription` — “Drift uses the camera so you can take a live verification selfie and add photos to your profile. Photos and the selfie are uploaded only when you confirm the upload.”
- `NSPhotoLibraryUsageDescription` — “Drift lets you pick existing photos for your profile.”
- `NSMicrophoneUsageDescription` — “Drift records a 30-second voice prompt for your profile so people can hear your voice before they wave.”
- `NSLocationWhenInUseUsageDescription` — “Drift uses your location only to fuzz it to a ZIP-prefix on your device. Drift’s servers never see your precise location.”
- `NSUserNotificationsUsageDescription` — “Drift notifies you when someone waves at you or sends a message.”

No Bluetooth strings, no precise-location strings.

Android — `AndroidManifest.xml`

- `INTERNET`
- `POST_NOTIFICATIONS`
- `RECEIVE_BOOT_COMPLETED`
- `CAMERA`
- `RECORD_AUDIO`
- `ACCESS_COARSE_LOCATION` (only if user opts into the ZIP-finder in onboarding; **never** `ACCESS_FINE_LOCATION`)

No Bluetooth permissions, no precise-location, no foreground service.

5. App Store / Play Store review notes

- The dating-app category requires demo credentials. Use a phone number we control (Twilio test number) and a verified profile flagged as `is_review_account` in Postgres.

- Verification image processing (AWS Rekognition CompareFaces) is documented in `PRIVACY.md` §3 and `SAFETY.md` §1. Highlight that we store **only the boolean result**, never the facial-feature embedding.
- Block + Report are surfaced in three places (profile, chat, settings) per Apple’s UGC guidelines.

6. Marketing site

`marketing.yml` deploys the static site at `index.html` to GitHub Pages. DNS for `drift.americangroupllc.com` (or whatever the brand domain is) is wired manually in the repo Settings → Pages → Custom domain.

7. What’s-new copy

Lives in `distribution/whatsnew/whatsnew-en-US/whatsnew.txt`. The Play release workflow uploads it via `r0adkll/upload-google-play@v1`. App Store Connect’s “What’s New” field is updated manually each release (automating it requires App Store Connect API privileges Drift doesn’t yet have).

Desktop binaries (Electron)

The `build-desktop` job in `.github/workflows/release.yml` uses `electron-builder` on an Ubuntu runner to produce three artifacts attached to every GitHub Release:

Artifact	Signed?	Notes
<code>Drift-Setup-X.Y.Z.exe</code> (NSIS)	no	Windows SmartScreen will prompt on first run; click <i>More info</i> → <i>Run anyway</i> .
<code>Drift-X.Y.Z.AppImage</code>	no	<code>chmod +x</code> then double-click on Linux.
<code>Drift-X.Y.Z.dmg</code>	no	macOS Gatekeeper will refuse. To install:

```
xattr -cr "/Volumes/Drift/Drift.app"
cp -R "/Volumes/Drift/Drift.app" /Applications/
```

(or *System Settings* → *Privacy & Security* → *Open Anyway*.)

A future change can add a separate `macos-latest` job + Apple Developer cert to produce a properly notarised .dmg without restructuring this workflow.

The Electron shell loads the existing root `index.html` in a `BrowserWindow` with `contextIsolation` enabled. See `desktop/main.js`.

TESTING

TESTING.md

Drift — TESTING

1. Test gate matrix

Layer	How to verify	Where it runs
Backend schema	<code>supabase db reset && supabase db diff</code> shows clean migration	CI <code>backend.yml</code>
Edge Function tests	<code>deno test</code> <code>backend/supabase/functions/**/_test.ts</code>	CI <code>backend.yml</code>
RLS test suite	hand-written SQL test that two distinct roles can/cannot see each other's profiles per layer	CI <code>backend.yml</code>
DriftCore Swift Package	<code>xcodebuild test -scheme DriftCore- Package -destination 'platform=iOS Simulator,name=iPhone 15,OS=latest'</code>	CI <code>ci.yml</code> + <code>ios.yml</code>
iOS app build	<code>xcodegen generate && xcodebuild build -sdk iphonesimulator</code>	CI <code>ios.yml</code>
iOS XCUITest smoke	mock Supabase → onboarding → discover → wave → chat → reply suggestion picker	CI <code>ios.yml</code>
watchOS build smoke	<code>xcodebuild build -sdk watchsimulator</code>	CI <code>ci.yml</code> + <code>ios.yml</code>
Android <code>:core</code>	<code>./gradlew :core:test</code> (Kotlin/JVM, NOT <code>:core:testDebugUnitTest</code>)	CI <code>ci.yml</code>
Android <code>:app</code>	<code>./gradlew :app:lintDebug</code> <code>:app:testDebugUnitTest</code> <code>:app:assembleDebug</code>	CI <code>ci.yml</code>
Wear <code>:wear</code>	<code>./gradlew :wear:testDebugUnitTest</code> <code>:wear:assembleDebug</code>	CI <code>ci.yml</code>
Android Compose UI smoke	<code>./gradlew</code> <code>:app:connectedDebugAndroidTest</code> walks discover → wave → chat	CI <code>android.yml</code>
Marketing site lint	<code>npx htmlhint</code> , <code>npx stylelint</code>	CI <code>ci.yml</code>

2. The four pure-logic keystone tests (XCTest and JUnit, mirrored)

The same scenarios run on both `shared/DriftCore/Tests/DriftCoreTests/` and
`android/core/src/test/java/com/americanagroupllc/drift/core/domain/`.

LayerScorer

- Same-ZIP, both verified, both `.dating` intent → score ≥ 0.85 .
- Different state, no shared interests → score < 0.30 .
- Tied scores break by `recent_activity` recency.
- Weighting follows: 30% intent, 20% layer, 15% interests, 15% verification, 10% recency, 10% conversation likelihood.

ToneClassifier

- 5-min gap between messages → `.slow`.
- 10 messages within 5 min → `.energetic`.
- Average message length > 200 chars → `.deep`.
- Pattern *"want to grab" / "meet up" / "coffee"* → `.meetupReady`.

LocationFuzzer

- Identical input never round-trips out of the function (no `Coordinate(lat:, lon:)` field on the output).
- ZIP prefix is always exactly 3 chars.
- Out-of-state coordinates resolve to a non-nil `stateCode`.
- Garbage input (e.g. lat=999) returns `nil` rather than throwing.

ReplyPromptBuilder

- Golden-output snapshot test for fixed inputs.
- Both profiles' `display_name`, `intent`, top vibe tag appear in the system prompt.
- Last 5 messages appear in chronological order (oldest → newest).
- Tone-specific clause appears when `tone == .meetupReady`.

These four files must compile + test green on **both** Swift and Kotlin before any UI work happens for that release.

3. Manual checks

Selfie verification end-to-end (real device)

- ☐ Open onboarding → Photos → upload 6 photos.
- ☐ Tag one as the verification reference.
- ☐ Open Verification → live preview detects face → tap capture.
- ☐ Edge Function logs show `CompareFaces  $\geq 0.9$` .
- ☐ Profile gets verified-checkmark; chat unlocks on next match.

Realtime chat (two devices)

- ☐ Device A and Device B match.
- ☐ A sends a message.

- ☐ B receives it via Realtime in < 1 s.

Layer fuzzing

- ☐ Enable Location on the client.
- ☐ Inspect Edge Function logs → only `zipPrefix3 / countyFips / stateCode` is present in any incoming POST. **No raw lat / lon**.

Push (iOS)

- ☐ Real iPhone receives a match notification.
- ☐ Rich-push preview shows the matched user's display name + thumbnail.
- ☐ Tap → opens to the conversation.

Push (Android)

- ☐ Real Android receives a match notification.
- ☐ Expanded notification shows quick-reply input.
- ☐ Quick-reply types into a draft and lands as a real message.

Watch complication (Apple Watch)

- ☐ Add complication to a watch face.
- ☐ Unread match count visible.
- ☐ Tap → quick-reply view.

Wear OS tile

- ☐ Add tile from Tile Carousel.
- ☐ Unread match count visible.
- ☐ Tap → quick-reply.

4. Coverage targets

- `shared/DriftCore/Sources/DriftCore/Domain/` — **100%** branch coverage. Pure logic, zero excuses.
- `shared/DriftCore/Sources/DriftCore/Networking/` — ≥ 70% line coverage with URLSession mocks.
- `android/core/src/main/java/com/americanagroupllc/drift/core/domain/` — **100%**.
- All other source ≥ 60% line coverage where reasonable; UI screens asserted by Compose UI / XCTest smokes.